Relatório 2

André Cota

Jhonatan Gutemberg

1464198@sga.pucminas.br

1202002@sga.pucminas.br

Link: Laboratório 2

Um Estudo das Características de Qualidade de Sistema Java

Belo Horizonte 2026

1. Resumo 2
1.1. Qual hipótese foi testada 2
1.2. Principais resultados 3
1.3. Decisão recomendada 3
2. Contexto e motivação 3
2.2. Por que esse experimento é necessário agora 3
2.3. Restrições e riscos 4
3. Objetivo e hipóteses 4
3.1. Objetivo 4
3.2. Hipótese nula (H0) e alternativa (H1) 4
3.3. Critérios de sucesso 5
4. Perguntas de pesquisa 5
4.1. Feedback Final das Revisões 5
4.2. Número de revisões 5
5. Variáveis e Métricas 5
5.1. Métricas de Qualidade 5
5.2. Tabela de métricas 6
1.1. Quadro Aceitação de PR 6
1.2. Quadro Número de revisões 6
5.3. Guardrails 7
6. Desenho experimental 7
7. Ambiente e materiais 7
8. Procedimento 7
9. Tratamentos e baseline 7
9.1. Baseline 7
9.2. Tratamento 7
10. Análise de dados 8
10.1. Método estatístico 8
10.2. Tratamento de outliers 8
11. Resultados 8
11.1. RQ01 – Popularidade x Qualidade: 8
11.2. RQ02 – Maturidade x Qualidade (Idade vs Qualidade) 8
11.3. RQ03 – Atividade vs Qualidade 9

11.4. RQ04 – Tamanho x Qualidade (LOC vs Qualidade) 9
12. Discussão e interpretação 9
12.1. Explicação 9
12.2. Limitações 9
12.3. Trade-offs 10
14. Decisão e recomendações 10
14.1. Decisão 10
15. Custos e impactos 10
15.1. Custo computacional 10
15.2. Risco técnico e operacional 10
16. Ética, privacidade e conformidade 10
17. Reprodutibilidade 10

1. Resumo

Este tópico sintetiza os objetivos, a abordagem experimental e os achados do estudo sobre a atividade de *code review* em repositórios populares do GitHub, sob a perspectiva da Engenharia de Software.

1.1. Qual hipótese foi testada

JKLJLGLKDJGDLJG

1.2. Principais resultados

- KJKLJKJDFDSKJFJDK

1.3. Decisão recomendada

Recomenda-se a implementação de políticas de **Small Pull Requests** nos fluxos de trabalho, visando reduzir o esforço cognitivo do revisor e acelerar o ciclo de *feedback*. Adicionalmente, as equipes devem monitorar PRs que ultrapassem as primeiras **48 horas** de análise sem uma decisão, pois o aumento desse tempo está estatisticamente correlacionado a uma maior probabilidade de rejeição e possível introdução de dívida técnica por baixa coesão.

2. Contexto e motivação

A prática de *code review* tornou-se uma etapa indispensável em processos de desenvolvimento ágil, servindo como uma barreira de qualidade antes da integração de código à base principal. No ecossistema do GitHub, essa atividade é centrada nos *Pull Requests* (PR), onde a interação entre desenvolvedores e revisores define a evolução do software.

2.1. Problema

O principal desafio reside na identificação de variáveis que influenciam o sucesso ou o fracasso de um *Pull Request*. Contribuições extensas ou complexas podem

sobrecarregar os revisores, levando a tempos de análise excessivos ou à rejeição do código (status `CLOSED`). Além disso, a falta de clareza sobre como o volume de interações e o tamanho das alterações afetam a qualidade interna do software dificulta a otimização dos fluxos de trabalho em projetos de grande escala.

2.2. Por que esse experimento é necessário agora

Com o crescimento acelerado de projetos *open source* e a dependência global de repositórios populares, entender a dinâmica das revisões é crucial para manter a saúde dos sistemas. Atualmente, o uso de ferramentas de automação e bots pode mascarar o comportamento humano nas revisões, tornando necessária uma análise que filtre apenas interações significativas (revisões de pelo menos uma hora). Este experimento é essencial para fornecer evidências baseadas em dados sobre como o comportamento das equipes impacta a manutenibilidade (acoplamento e coesão) em um cenário de alta rotatividade de contribuições.

2.3. Restrições e riscos

A execução deste experimento enfrenta as seguintes limitações e riscos:

- **Cotas da API**: O GitHub impõe limites rígidos de taxa (*rate limits*) para consultas GraphQL, o que pode interromper a coleta de dados em larga escala.
- **Viés de Amostragem**: Embora a amostragem estatística reduza o tempo de processamento, existe o risco de não capturar comportamentos atípicos de repositórios específicos se a amostra não for adequadamente aleatória.
- **Qualidade dos Metadados**: A análise depende da consistência das informações preenchidas pelos desenvolvedores nos campos de `bodyText` e `state`, que podem variar entre diferentes culturas de projetos.
- **Filtros de Bots**: O risco de incluir revisões automáticas no dataset é mitigado pelo filtro de tempo mínimo (1 hora), porém revisões humanas extremamente rápidas podem ser erroneamente descartadas.

3. Objetivo e hipóteses

3.1. Objetivo

O objetivo deste laboratório é analisar a atividade de code review em repositórios populares e de código aberto no GitHub, a fim de identificar quais variáveis de processo influenciam o resultado final de uma submissão de código (Pull Request) e

o esforço despendido na sua revisão.

3.2. Hipótese nula (H0) e alternativa (H1)

A hipótese tem como objetivo é responder como variáveis de processo (atividades de revisão, tempo e interações) influenciam a qualidade interna do software.

- **Hipótese Nula (H0):** Não há correlação significativa entre as métricas de processo (tamanho do PR, tempo de análise e interações) e as métricas de qualidade interna do código submetido.
- **Hipótese Alternativa (H1):** Repositórios com maior atividade de revisão ou PRs com grandes alterações de tamanho tendem a apresentar maior acoplamento e menor coesão, indicando que a alta rotatividade ou volume de código impacta negativamente na manutenibilidade.

3.3. Critérios de sucesso

SERÁ AVALIADO

4. Perguntas de pesquisa

4.1. Feedback Final das Revisões

- RQ 01. Qual a relação entre o tamanho dos PRs e o feedback final das revisões?
- RQ 02. Qual a relação entre o tempo de análise dos PRs e o feedback final das revisões?
- RQ 03. Qual a relação entre a descrição dos PRs e o feedback final das revisões?
- RQ 04. Qual a relação entre as interações nos PRs e o feedback final das revisões?

4.2. Número de revisões

- RQ 05. Qual a relação entre o tamanho dos PRs e o número de revisões realizadas?
- RQ 06. Qual a relação entre o tempo de análise dos PRs e o número de revisões
- realizadas?
- RQ 07. Qual a relação entre a descrição dos PRs e o número de revisões realizadas?

RQ 08. Qual a relação entre as interações nos PRs e o número de revisões realizadas?

5. Variáveis e Métricas

5.1. Métricas de Qualidade

- Tamanho: número de arquivos; total de linhas adicionadas e removidas.
- Tempo de Análise: intervalo entre a criação do PR e a última atividade (fechamento ou merge).
- Descrição: número de caracteres do corpo de descrição do PR (na versão markdown).
- Interações: número de participantes; número de comentários.

5.2. Tabela de métricas

1.1. Quadro Aceitação de PR

Questão de Pesquisa	Métrica	Justificativa
RQ01	`Quantidade de linhas adicionadas`, `Quantidade de linhas deletadas`, `Quantidade arquivos alterados`, `tamanho dos arquivos`	PRs grandes são complexos e difíceis de inspecionar.
RQ02	Tempo de análise	PRs problemáticos tendem a ficar abertos por mais tempo.
RQ03	Tamanho da mensagem	Descrições detalhadas facilitam o entendimento do revisor.
RQ04	`participants`, `comments`, `interactions`	Maior colaboração pode indicar refinamento da solução.

1.2. Quadro Número de revisões

Questão de Pesquisa	Métrica	Justificativa

RQ05	`size, reviews`	Mudanças extensas demandam maior esforço de revisão.
RQ06	`analysis_time_h, reviews`	O aumento de revisões amplia o tempo total de análise.
RQ07	`body_len, reviews`	Descrições vagas podem gerar ciclos extras de revisão.
RQ08	`interactions, reviews`	Mais interações costumam indicar ciclos de revisão maiores.

5.3. Guardrails

- LOC
- Quantidade mínima de PR aceitas

6. Desenho experimental

- Tipo: Benchmark
- Unidades: Repositórios GitHub.
- Tamanho de amostra: 1000 Repositórios Java
- Duração e janela temporal: Dados coletados até a data presente, considerando o histórico total dos repositórios.

7. Ambiente e materiais

- Hardware: Intel(R) Core (TM) i3-8130U CPU @ 2.20GHz (2.21 GHz)
- Software: VSCode, Ferramenta CK
- Tecnologias: Java, Python
- Dataset : dataset_final.csv contendo as métricas agregadas por mediana/média.

8. Procedimento

1. Consulta à API do GitHub para listar os 1.000 repositórios Java.
2. Script de automação para realizar o *git clone* de cada projeto.

3. Execução da ferramenta CK em cada repositório para extração de métricas de classe.
4. Agregação dos dados (média/mediana por projeto) no arquivo CSV final.
5. Geração de gráficos de dispersão e matriz de correlação.

9. Tratamentos e baseline

9.1. Baseline

Projetos de pequeno porte e baixa atividade.

9.2. Tratamento

Comparação do comportamento das métricas de qualidade conforme as variáveis de processo aumentam (escala).

10. Análise de dados

10.1. Método estatístico

A análise estatística foi realizada utilizando o Coeficiente de Correlação de Pearson, através da biblioteca Pandas, para medir a associação linear entre as variáveis de processo (estrelas, idade, releases, LOC) e as métricas de qualidade (CBO, DIT, LCOM).

10.2. Tratamento de outliers

Remoção de repositórios com LOC < 10 (geralmente documentações ou exercícios simples).

11. Resultados

11.1. RQ01 – Popularidade x Qualidade:

Ao analisar a relação entre a popularidade (número de estrelas) e a qualidade interna, os dados mostram uma ausência de correlação linear.

- A correlação entre Popularidade e Coesão (LCOM) apresentou um valor de 0,047, o que é estatisticamente insignificante.
- A correlação entre Popularidade e Acoplamento (CBO) apresentou um valor de -0,041.

Os valores próximos de zero confirmam a Hipótese Nula (H0), demonstrando que o fato de um projeto ser muito popular no GitHub não garante que ele possua um design de código superior ou mais coeso.

11.2. RQ02 – Maturidade x Qualidade (Idade vs Qualidade)

Ao correlacionar a idade do repositório (age_years) com as métricas de qualidade, observa-se se o tempo de existência do projeto degrada o código.

Resultado: A correlação com CBO e LCOM costuma ser baixa/neutra (geralmente entre 0.05 e 0.15).

A maturidade isolada não é um fator de degradação. Projetos antigos podem ser bem mantidos (refatorados) ou simplesmente estarem estáveis.

11.3. RQ03 – Atividade vs Qualidade

Esta questão busca entender se a frequência de atualizações (medida pelo número de releases) impacta a saúde interna do software.

Resultados Numéricos:

- **Releases vs CBO (Acoplamento)**: Apresenta uma correlação positiva leve (geralmente entre 0.15 e 0.25).
- **Releases vs LCOM (Falta de Coesão)**: Apresenta correlação baixa (próxima de 0.10).
- **Releases vs DIT (Herança)**: Correlação quase nula.

O gráfico de dispersão mostra que a maioria dos repositórios se concentra em um volume menor de releases (0 a 50), mas mesmo nos projetos com alta atividade (centenas de releases), o acoplamento (CBO) não dispara descontroladamente, mantendo uma tendência de crescimento suave.

11.4. RQ04 – Tamanho x Qualidade (LOC vs Qualidade)

O gráfico abaixo revela uma tendência de crescimento clara: quanto maior o LOC, maior o LCOM (perda de coesão).

12. Discussão e interpretação

12.1. Explicação

Sistemas maiores tendem a acumular "classes divinas" (God Classes), o que explica o aumento do LCOM. A popularidade é um fenômeno social, não técnico.

12.2. Limitações

A análise não diferencia bibliotecas (libs) de aplicações finais, que possuem padrões de acoplamento distintos.

12.3. Trade-offs

Projetos com muitas releases (alta atividade) ganham funcionalidade, mas pagam o preço em acoplamento (CBO) crescente.

14. Decisão e recomendações

14.1. Decisão

Iterar. Recomenda-se que grandes projetos adotem políticas estritas de quebra de classes após certo limiar de LOC.

14.2. Monitoramento pós-lançamento

Integrar a métrica LCOM ao Dashboard de CI dos projetos analisados.

15. Custos e impactos

15.1. Custo computacional

Aproximadamente 6 horas de processamento para clonar e analisar 1.000 repositórios.

15.2. Risco técnico e operacional

Dependência da disponibilidade da API do GitHub e integridade dos códigos-fonte (alguns projetos podem não compilar).

16. Ética, privacidade e conformidade

Os dados são públicos sob licenças open-source. Nenhuma informação pessoal de desenvolvedores foi processada além dos metadados públicos dos repositórios.

17. Reprodutibilidade

Para que seja possível reproduzir o experimento, basta clonar nosso repositório do Github, criar um token do próprio Githubs e executar o main.py.